

Task 1

Know your customers

Take-home assessment

Roughly two-thirds of your total time budget goes here.

We run the marketplace pharmacies use to order medicines from many suppliers in one place. Every supplier records the same pharmacy in its own system, under its own name, with its own customer number, and with its own idea of where that pharmacy is. What you will receive is a sanitized extract of exactly that. It is not curated and it is not clean. Your job is to turn it into one answer to a simple question: **who are our customers, where are they, and how much do they buy?**

Ground rules

1. **Offline code only.** No network calls, no model APIs inside submitted code. Any offline library is fine.
2. **One command reruns everything.** Every number you report must come from code you hand us, deterministically.
3. **Honesty about gaps.** Partial and true beats complete and invented.
4. **Ask, then proceed.** If a definition matters and this document is silent, ask early, state your assumption, keep moving.
5. **Trust nothing blindly** — including the data, and your own first results.
6. Using AI assistants is allowed and expected; ship an honest `AI_DISCLOSURE.md`. Fabricating results or disclosure ends the conversation.

The data

UTF-8 (BOM) CSVs inside `dataset/`. Column meanings are yours to establish: some are self-evident, some you confirm by testing against the data, and a few only reveal themselves through connections between files. A `_manifest.json` holds row counts and checksums. Window: September 2024 to August 2026. Currency: EGP.

File	Contents
------	----------

pharmacy_registry.csv	our master record of client pharmacies
areas_reference.csv	the list of areas we operate in, with map centers
supplier_account_names.csv	each supplier branch's own naming book for customer accounts
supplier_branches.csv	branches, order minimums, payment terms, discount conventions
invoices_app.csv / invoice_details_app.csv	orders placed in our mobile app
invoices_erp.csv / invoice_details_erp.csv	orders billed through a partner ERP
invoices_legacy.csv / invoice_details_legacy.csv	the oldest system; also where credit documents live

Your submission

All of your work lives on your fork of this repository (a task1/ folder with code, requirements.txt, an entry point we can run from a clean folder, your README, AI_DISCLOSURE.md, and the deliverables below). Commit as you go; the deliverable is your fork link.

Deliverables

1. **The identity puzzle.** Every order belongs to exactly one pharmacy in the registry. App and legacy rows expose ids in some form. ERP rows know only an account name as typed by the supplier branch that billed it — and every branch keeps its own naming book, with its own customer numbers, sometimes written differently on every export. Reconcile all of it into answers/entity_map.csv.

Expected output (shape, not answers). columns account_name_raw, branch_code, resolved_to, confidence, method_note; resolved_to is a registry id or an explicit exclusion bucket with a reason. In the README: bucket counts, and for your five highest-spend pharmacies the clean canonical name plus every alias that fed into it. Unresolvable text stays unresolved — a measured unknown beats a forced guess, and we check.

2. **The location puzzle.** A large share of registry rows lost their geography. The invoices remember it though: ERP headers carry scattered, sometimes wrong area fields and vague free-text addresses, while ERP *line rows* carry the delivery address as the driver would read it. Mine those, reconcile them with the rows that still know where they are, and produce answers/pharmacy_locations.csv.

Expected output (shape, not answers). columns `pid`, `area`, `lat`, `lon`, `confidence`, `evidence_count`. Rows whose area was already known in the registry must agree with it. For the rest: fill what the evidence supports, leave the rest honestly empty, and explain your method in one paragraph. We will grade your recovered areas against ours.

3. **The canonical ledger.** `data/canonical_lines.csv` — one row per valid line item across all sources, with at least `pid`, `date`, `source`, `qty`, `unit_price`, `discount_value`, `net_value`. Sources disagree about what counts as revenue; your policies (counted / excluded / negated) are yours to define and defend with row counts.

Expected output (shape, not answers). policy table in the README: rule, rows affected, EGP impact.

4. **The column dictionary.** `answers/column_dictionary.md`: meaning, units, domain values for every column of every file — each backed by evidence you actually ran. Flag every column you found unreliable, with the condition under which it misleads.

Expected output (shape, not answers). per-file sections; every unreliable-column entry cites a check anyone can rerun.

5. **The dossier.** From YOUR ledger and locations (`answers/business_dossier.md`), answer exactly these, all with tables:

- (a) How many pharmacies operate in each area?
- (b) How much revenue did each area generate? (top-10 by revenue)
- (c) Which pharmacies generated the most revenue? (top-10 overall)
- (d) Within each of those top-10 areas: its own top-10 pharmacies by revenue.
- (e) Which suppliers generated the most revenue? (top-10)

Expected output (shape, not answers). ≤ 4 pages; every figure points into your ledger; areas without full location evidence are reported with their confidence, not silently dropped.

6. **The analysis website.** We do not read static reports for a living. Build a small local web application on top of YOUR cleaned data that lets us analyze what you produced: browse pharmacies, areas and suppliers; filter by date range and by area; see the rankings and the numbers behind them; drill into one pharmacy and see its aliases, its recovered location and its revenue over time.

Expected output (shape, not answers). runs locally from your repository with one command (a README section “how to run the site”); every table on the site is computed from your ledger, not hardcoded.

Stack. We suggest Streamlit — it is the fastest path to this under a time budget — but we accept any web stack you are comfortable with (Flask, Django, plain HTML+JS, whatever). Python-, browser- and offline-friendly wins.

What “good” means. A ledger and a map built on sources you first made trustworthy, with every ambiguity either measured or asked about. There is no single correct cleaning rule set — there are defensible ones and indefensible ones, and the difference is documentation.